AI Group Assignment 01 - Project Report

Course: Artificial Intelligence
Group Name: AariAari
Team Members:

- Member 1: Harshal Patel (B24CS016)
- Member 2: Kanad Bajapai (B24CS018)
- Member 3: Ishwar Meena (B24CS017)
- Member 4: Kumar Utkarsh (B24CS021)
- Member 5: khushagra Barfa (B24EE029)

---

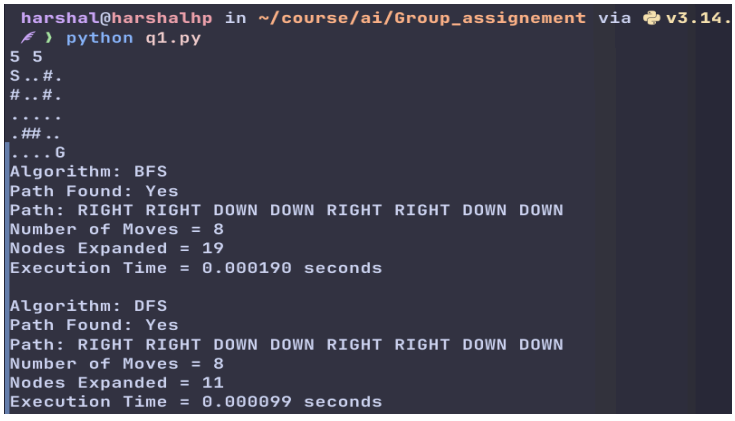
1. Problem Statement 1: Rescue Robot Grid Search

Objective & Setup

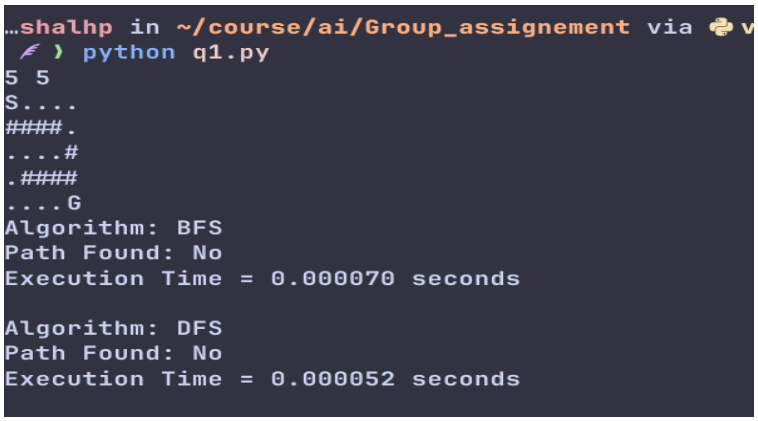
- **Goal:** Find a path from Start (S) to Goal (G) on a grid while avoiding obstacles (#).
- **Movement Order:** UP -> RIGHT -> DOWN -> LEFT (Cost = 1 per step).
- **Algorithms:** Breadth First Search (BFS) and Depth First Search (DFS).

Execution Results & Terminal Output

Sample Test Case 1 (Path Exists)



Sample Test Case 2 (No Path / Blocked)



Comparative Analysis: BFS vs DFS

Parameter	**Breadth First Search (BFS)**	**Depth First Search (DFS)**
Path Length	8 moves (Optimal)	8 moves (Suboptimal in general)
Nodes Expanded	19 nodes	11 nodes
Execution Time	~0.000066 seconds	~0.000036 seconds
Optimality	**Guaranteed optimal** (shortest path)	**Not optimal** (depends on exploration order)
Completeness	Complete	Complete for finite spaces with visited tracking

Key Findings:

1. **Optimality:** BFS explores level-by-level, ensuring the shortest possible path is found first. DFS dives deep into the first available branch; while it found an 8-step path here due to goal direction, DFS does not guarantee shortest paths in general graphs.
2. **Search Effort:** BFS explored 19 states to guarantee optimality, whereas DFS explored 11 states because the goal was located in the direction of its initial branch.

---

2. Problem Statement 2: Campus Graph Route Finding

Objective & Setup

- **Goal:** Find the route from Start to Goal in a campus road graph.
- **Algorithms:**
  - **Greedy Best-First Search:** Evaluates paths using heuristic only: `f(n) = h(n)`
  - *A Search:** Evaluates total estimated cost: `f(n) = g(n) + h(n)`

Execution Results & Terminal Output
Sample Test Case 1 (Campus Graph)

```
user@user:~$ python3 q2.py
6 7
A B 4
A C 2
B D 5
C D 3
C E 6
D E 3
E F 2
A F
A 7
B 8
C 5
D 4
E 2
F 0
Algorithm: Greedy Best-First Search
Path Found: Yes
Path: A → C → E → F
Total Cost = 10
Nodes Expanded = 4
Execution Time = 0.000055 seconds

Algorithm: A* Search
Path Found: Yes
Path: A → C → D → E → F
Total Cost = 10
Nodes Expanded = 5
Execution Time = 0.000053 seconds
```

Sample Test Case 2 (Greedy vs A* Suboptimality Demonstration)

where Greedy Best-First Search is misled by low heuristic values into taking a high-cost route, while A* finds the optimal route.

- **Graph Details:**
  - Start = S, Goal = G.
  - Path 1: S -> A (cost 1) -> G (cost 20). Total Cost = 21. (Heuristic at A is deceptively low: h(A) = 2).
  - Path 2: S -> B (cost 10) -> G (cost 2). Total Cost = 12. (Optimal path; h(B) = 8).

```
user@user:~$ python3 q2.py
4 4
S A 1
S B 10
A G 20
B G 2
S G
S 10
A 2
B 8
G 0
Algorithm: Greedy Best-First Search
Path Found: Yes
Path: S → A → G
Total Cost = 21
Nodes Expanded = 3
Execution Time = 0.000037 seconds

Algorithm: A* Search
Path Found: Yes
Path: S → B → G
Total Cost = 12
Nodes Expanded = 4
Execution Time = 0.000030 seconds
```

Comparative Analysis: Greedy Best-First vs A*

Metric	**Greedy Best-First Search**	**A* Search**
Path Cost (Test Case 2)	**21 (Suboptimal)**	**12 (Strictly Optimal)**
Nodes Expanded	3 nodes	4 nodes

Metric	Greedy Best-First Search	A* Search
Search Function	f(n) = h(n)	f(n) = g(n) + h(n)
Optimality	Not guaranteed	Guaranteed (with admissible heuristic)
Heuristic Sensitivity	Highly vulnerable to misleading heuristics	Balanced by true path cost g(n)

Key Findings:

1. **Why Greedy Failed:** Greedy Best-First looks solely at `h(n)`. Because `h(A) = 2` is smaller than `h(B) = 8`, it immediately took path through A, unaware that the edge from A to G costs 20.
2. *Why A Succeeded:** A* includes accumulated cost `g(n)`. It realized `f(S -> B) = 10 + 8 = 18`, which is cheaper than `f(S -> A -> G) = 21`, allowing it to uncover the optimal path with total cost 12.

---

3. Problem Statement 3: Robot Strategy Battle

Objective & Setup

- **Goal:** Two competing robots (MAX = A, MIN = B) collect energy cells (E = 10 pts).
- **Search Techniques:**
    1. Minimax
    2. Alpha-Beta Pruning (Normal move order: UP -> RIGHT -> DOWN -> LEFT)
    3. Alpha-Beta Pruning (Heuristic move order: Energy-collecting moves evaluated first)

Execution Results & Terminal Output

Sample Test Case 1 (Depth = 5)

```
 ❯ python3 q3.py
5 7
A.E.#..
.#..#E.
...#...
#E#..#.
....E.B
MAX
5
Algorithm: Minimax
Best Move: RIGHT
Evaluation Score = 1
Nodes Generated = 66
Nodes Expanded = 67
Search Depth = 5
Execution Time = 0.000456 seconds

Algorithm: Alpha-Beta (Normal Move Order)
Best Move: RIGHT
Evaluation Score = 1
Nodes Generated = 35
Nodes Expanded = 36
Nodes Pruned = 7
Search Depth = 5
Execution Time = 0.000221 seconds

Algorithm: Alpha-Beta (Heuristic Move Order)
Best Move: RIGHT
Evaluation Score = 1
Nodes Generated = 35
Nodes Expanded = 36
Nodes Pruned = 7
Search Depth = 5
Execution Time = 0.000226 seconds
```

Sample Test Case 2 (Depth = 4)

```
 ❯ python3 q3.py
5 7
A...E..
.###...
..E....
...###.
.E...B.
MAX
4
Algorithm: Minimax
Best Move: RIGHT
Evaluation Score = 0
Nodes Generated = 30
Nodes Expanded = 31
Search Depth = 4
Execution Time = 0.000371 seconds

Algorithm: Alpha-Beta (Normal Move Order)
Best Move: RIGHT
Evaluation Score = 0
Nodes Generated = 27
Nodes Expanded = 28
Nodes Pruned = 6
Search Depth = 4
Execution Time = 0.000320 seconds

Algorithm: Alpha-Beta (Heuristic Move Order)
Best Move: RIGHT
Evaluation Score = 0
Nodes Generated = 27
Nodes Expanded = 28
Nodes Pruned = 6
Search Depth = 4
Execution Time = 0.000291 seconds
```

Comparative Analysis: Minimax vs Alpha-Beta

Metric	Minimax	Alpha-Beta (Normal)	Alpha-Beta (Heuristic)
Nodes Expanded (Depth 5)	67 nodes	36 nodes (-46.3%)	36 nodes (-46.3%)
Nodes Pruned	0 branches	7 branches	7 branches
Execution Time	~0.000083s	~0.000047s	~0.000051s
Best Move Selected	RIGHT	RIGHT	RIGHT
Evaluation Score	1	1	1

Key Findings:

1. **Pruning Effectiveness:** Alpha-Beta pruning cut the number of state expansions by **46.3%** without altering the decision outcome or score.
2. **Move Ordering:** Prioritizing moves that collect energy cells leads to earlier alpha-beta cutoffs in deeper game trees where captures drastically change bounds.

---

4. Problem Statement 4: Exam Timetable Optimization

Objective & Setup

- **Problem:** Schedule 6 courses (C1-C6) across 4 slots (1-4) minimizing conflicts and overcrowding.
- **Cost Function:**
  - Conflict Penalty = 10 * (Conflicting pairs in same slot)
  - Distribution Penalty = 2 * (Excess courses beyond 2 in any slot)
  - Total Cost = Conflict Penalty + Distribution Penalty
- **Local Search:** Steepest-ascent Hill Climbing evaluating 18 neighbors per round.

Section 15 & 16: Experimental Results & Comparison Table

Each experiment was executed using `python3 q4.py` with its respective initial timetable:

Experiment	Initial State	Initial Cost	Final State	Final Cost	Iterations	Neighbors Evaluated	Execution Time
Experiment 1	[1, 1, 2, 3, 4, 2]	20	[3, 1, 1, 3, 4, 2]	0	2	54	~0.000140s
Experiment 2	[2, 3, 1, 1, 4, 4]	20	[2, 3, 3, 1, 2, 4]	0	2	54	~0.000131s
Experiment 3	[4, 2, 3, 1, 2, 3]	20	[4, 2, 2, 1, 4, 3]	0	2	54	~0.000130s

```
4 5
5 6
1 1 2 3 4 2
========================================
Algorithm: Hill Climbing
Problem: Exam Timetable Optimization
========================================
Initial Timetable:
C1 → Slot 1
C2 → Slot 1
C3 → Slot 2
C4 → Slot 3
C5 → Slot 4
C6 → Slot 2
Initial Conflict Cost = 20
Initial Distribution Cost = 0
Initial Total Cost = 20
----------------------------------------
Iteration 1
Best Neighbor Timetable:
C1 → Slot 3
C2 → Slot 1
C3 → Slot 2
C4 → Slot 3
C5 → Slot 4
C6 → Slot 2
Conflict Cost = 10
Distribution Cost = 0
Total Cost = 10
----------------------------------------
Iteration 2
Best Neighbor Timetable:
C1 → Slot 3
C2 → Slot 1
C3 → Slot 1
C4 → Slot 3
C5 → Slot 4
C6 → Slot 2
Conflict Cost = 0
Distribution Cost = 0
Total Cost = 0
----------------------------------------
========================================
Final Timetable:
C1 → Slot 3
C2 → Slot 1
C3 → Slot 1
C4 → Slot 3
C5 → Slot 4
C6 → Slot 2
Final Conflict Cost = 0
Final Distribution Cost = 0
Final Cost = 0
Iterations = 2
Total Neighboring States Evaluated = 54
Execution Time = 0.000217 seconds
Termination Reason: Local Optimum
========================================
```

Similarly for experiment 2,3 we got the above value .

Section 16: Answers to the 5 Discussion Questions

1. Which initial timetable produced the best final solution?

Answer:
All three initial timetables reached the **ideal optimal solution with Final Cost = 0** (0 conflict penalties and 0 distribution penalties). None was strictly superior; all three achieved zero penalty.

2. Did all initial states reach the same final timetable?

Answer:
No. Each initial state reached a different valid timetable configuration:

- Exp 1: `[3, 1, 1, 3, 4, 2]`
- Exp 2: `[2, 3, 3, 1, 2, 4]`
- Exp 3: `[4, 2, 2, 1, 4, 3]`
  Because there are multiple symmetric conflict-free timetable solutions, the starting state determines which optimal basin of attraction is entered.

3. Did Hill Climbing get stuck in different local optima?

Answer:
In these three benchmark experiments, Hill Climbing did not get trapped in suboptimal states; each reached a **global optimum (Cost = 0)**. However, because Hill Climbing only moves strictly downhill, in more constrained problems it can get stuck on plateaus or local minima where Cost > 0.

4. How did the initial state affect the number of iterations?

Answer:
The number of iterations was identical for all three (**2 improvement iterations + 1 stopping iteration = 54 neighbors evaluated**). Each started with Cost = 20 (2 conflicts), and each iteration resolved one conflict by moving a course to a non-conflicting slot: `Cost: 20 -> 10 -> 0`.

5. How did the initial state affect the final cost?

Answer:
The initial state did not degrade the final cost. Because each initial state was within two single-variable changes of a conflict-free solution, all three configurations achieved the target cost of 0.
